

Anàlisi_complert_de_Dataset_de_Diamonds

December 15, 2025

0.1 Anàlisi complert dataset diamonds

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score,
↳GridSearchCV
from sklearn.preprocessing import StandardScaler, PolynomialFeatures,
↳OrdinalEncoder, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

1. Carregar el dataset:

```
[2]: diamonds = sns.load_dataset('diamonds')
#passar-ho a DataFrame
diamonds = pd.DataFrame(diamonds)
```

2. EDA (informació general, estadístiques, nuls, duplicats, etc.)

```
[3]: diamonds.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53940 entries, 0 to 53939
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   carat        53940 non-null  float64
1   cut          53940 non-null  category
2   color        53940 non-null  category
3   clarity      53940 non-null  category
4   depth        53940 non-null  float64
5   table        53940 non-null  float64
6   price        53940 non-null  int64  
7   x            53940 non-null  float64
8   y            53940 non-null  float64
9   z            53940 non-null  float64
dtypes: category(3), float64(6), int64(1)
```

memory usage: 3.0 MB

```
[4]: diamonds.head()
```

```
[4]:   carat      cut color clarity depth  table  price     x     y     z
     0   0.23   Ideal     E    SI2   61.5   55.0    326   3.95   3.98   2.43
     1   0.21  Premium     E    SI1   59.8   61.0    326   3.89   3.84   2.31
     2   0.23    Good     E    VS1   56.9   65.0    327   4.05   4.07   2.31
     3   0.29  Premium     I    VS2   62.4   58.0    334   4.20   4.23   2.63
     4   0.31    Good     J    SI2   63.3   58.0    335   4.34   4.35   2.75
```

```
[5]: diamonds.describe()
```

```
[5]:
```

	carat	depth	table	price	x \
count	53940.000000	53940.000000	53940.000000	53940.000000	53940.000000
mean	0.797940	61.749405	57.457184	3932.799722	5.731157
std	0.474011	1.432621	2.234491	3989.439738	1.121761
min	0.200000	43.000000	43.000000	326.000000	0.000000
25%	0.400000	61.000000	56.000000	950.000000	4.710000
50%	0.700000	61.800000	57.000000	2401.000000	5.700000
75%	1.040000	62.500000	59.000000	5324.250000	6.540000
max	5.010000	79.000000	95.000000	18823.000000	10.740000

	y	z
count	53940.000000	53940.000000
mean	5.734526	3.538734
std	1.142135	0.705699
min	0.000000	0.000000
25%	4.720000	2.910000
50%	5.710000	3.530000
75%	6.540000	4.040000
max	58.900000	31.800000

```
[6]: diamonds.isnull().sum()
```

```
[6]: carat      0
     cut      0
     color    0
     clarity  0
     depth    0
     table    0
     price    0
     x        0
     y        0
     z        0
     dtype: int64
```

```
[7]: diamonds.duplicated().sum()
diamonds[diamonds.duplicated(keep=False)]
```

```
[7]:
```

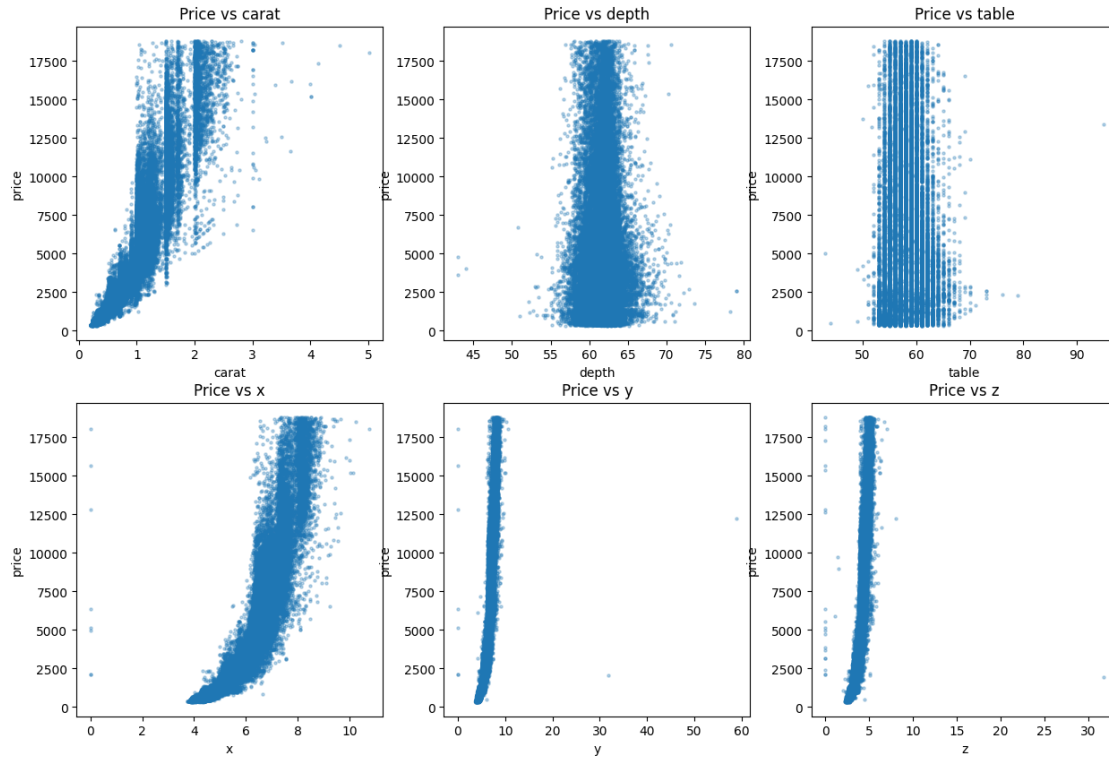
	carat	cut	color	clarity	depth	table	price	x	y	z
1004	0.79	Ideal	G	SI1	62.3	57.0	2898	5.90	5.85	3.66
1005	0.79	Ideal	G	SI1	62.3	57.0	2898	5.90	5.85	3.66
1006	0.79	Ideal	G	SI1	62.3	57.0	2898	5.90	5.85	3.66
1007	0.79	Ideal	G	SI1	62.3	57.0	2898	5.90	5.85	3.66
1008	0.79	Ideal	G	SI1	62.3	57.0	2898	5.90	5.85	3.66
...	...	...	...	...	...	...	...	...	...	...
49557	0.71	Good	F	SI2	64.1	60.0	2130	0.00	0.00	0.00
50078	0.51	Ideal	F	VVS2	61.2	56.0	2203	5.19	5.17	3.17
50079	0.51	Ideal	F	VVS2	61.2	56.0	2203	5.19	5.17	3.17
52860	0.50	Fair	E	VS2	79.0	73.0	2579	5.21	5.18	4.09
52861	0.50	Fair	E	VS2	79.0	73.0	2579	5.21	5.18	4.09

[289 rows x 10 columns]

3. Primera visualització de relació entre dades numèriques

```
[8]: numeric_features = ['carat', 'depth', 'table', 'x', 'y', 'z']
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.ravel()

for idx, feature in enumerate(numeric_features):
    axes[idx].scatter(diamonds[feature], diamonds['price'], alpha=0.3, s=5)
    axes[idx].set_xlabel(feature)
    axes[idx].set_ylabel('price')
    axes[idx].set_title(f'Price vs {feature}')
```



4. Regressió Lineal amb tots els “features”

```
[9]: X_numeric = diamonds[numeric_features]
y = diamonds['price']

X_train, X_test, y_train, y_test = train_test_split(
    X_numeric, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
```

5. Avaluació dels resultats

```
[10]: print(f"Model amb TOTES les variables:")
print(f"    R² Train: {lr.score(X_train, y_train):.4f}")
print(f"    R² Test:  {r2_score(y_test, y_pred_lr):.4f}")
print(f"    MSE Test: {mean_squared_error(y_test, y_pred_lr):.2f}")
```

```
print(f" MAE Test: {mean_absolute_error(y_test, y_pred_lr):.2f}")
```

Model amb TOTES les variables:

R² Train: 0.8593

R² Test: 0.8590

MSE Test: 2242178.90

MAE Test: 888.48

```
[11]: lr.coef_
```

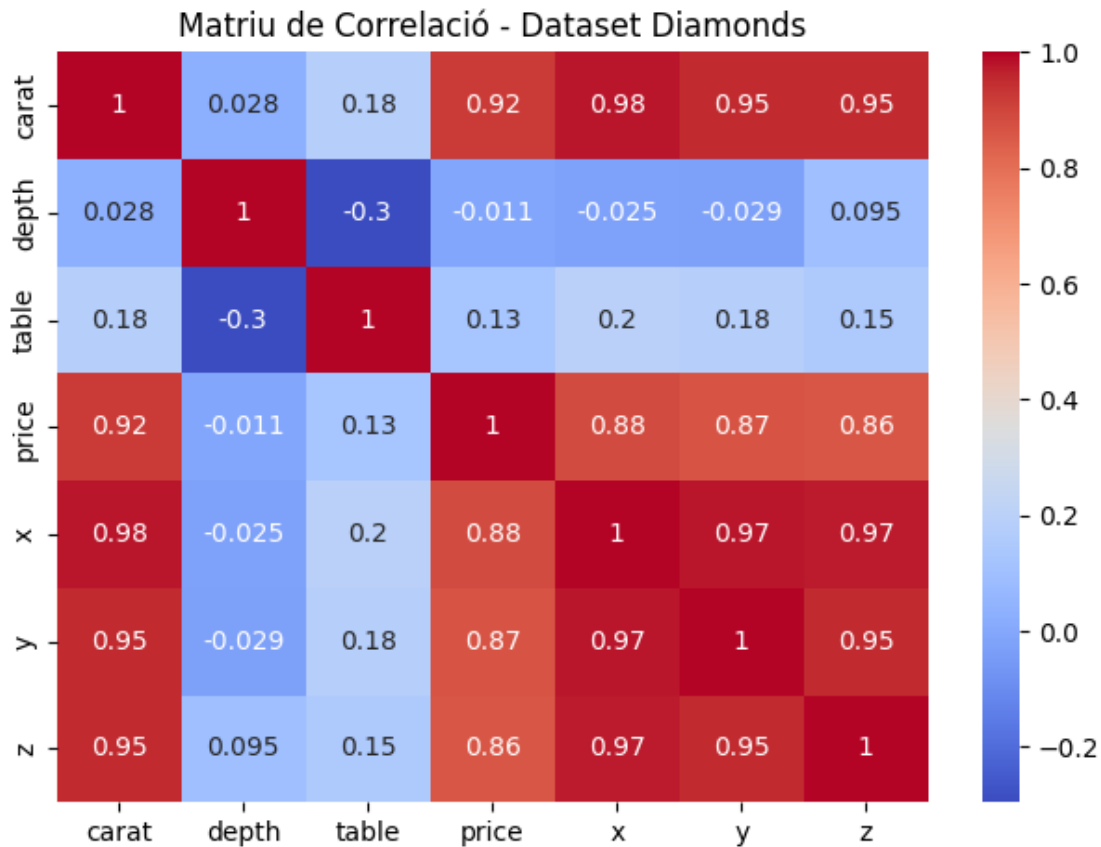
```
[11]: array([10683.18425966, -204.09955335, -104.26736872, -1286.79957429,  
          37.63252069, 53.39486424])
```

6. Matriu de correlació per obtenir els millors 'features'

```
[12]: diamonds_num = diamonds.drop(['cut', 'color', 'clarity'], axis=1)  
correlation_matrix = diamonds_num.corr()  
print(correlation_matrix)
```

	carat	depth	table	price	x	y	z
carat	1.000000	0.028224	0.181618	0.921591	0.975094	0.951722	0.953387
depth	0.028224	1.000000	-0.295779	-0.010647	-0.025289	-0.029341	0.094924
table	0.181618	-0.295779	1.000000	0.127134	0.195344	0.183760	0.150929
price	0.921591	-0.010647	0.127134	1.000000	0.884435	0.865421	0.861249
x	0.975094	-0.025289	0.195344	0.884435	1.000000	0.974701	0.970772
y	0.951722	-0.029341	0.183760	0.865421	0.974701	1.000000	0.952006
z	0.953387	0.094924	0.150929	0.861249	0.970772	0.952006	1.000000

```
[13]: sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')  
plt.title('Matriu de Correlació - Dataset Diamonds')  
plt.tight_layout()  
plt.show()
```



Podem veure com: * Carat té la correlació més alta amb price (0.92) * X, Y, Z (dimensions) també molt correlacionades amb price * X, Y, Z estan molt correlacionades entre elles (multicolinearitat) * Depth i Table tenen correlació baixa amb price”

```
[14]: best_features = ['carat', 'x']
```

```
[15]: X_numeric = diamonds[best_features]
y = diamonds['price']

X_train, X_test, y_train, y_test = train_test_split(
    X_numeric, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
mae_lr = mean_absolute_error(y_test, y_pred_lr)
```

```
r2_lr = r2_score(y_test, y_pred_lr)
```

```
[16]: print(f"Model amb TOTES les variables:")
print(f" R² Train: {lr.score(X_train, y_train):.4f}")
print(f" R² Test: {r2_score(y_test, y_pred_lr):.4f}")
print(f" MSE Test: {mean_squared_error(y_test, y_pred_lr):.2f}")
print(f" MAE Test: {mean_absolute_error(y_test, y_pred_lr):.2f}")
```

Model amb TOTES les variables:

R² Train: 0.8535
R² Test: 0.8532
MSE Test: 2334040.06
MAE Test: 915.52

7. Codifiquem els valors categòrics i normalitzem els valors del DataFrame

```
[17]: categorical_cols = ['cut', 'color', 'clarity']

cut_order = ['Fair', 'Good', 'Very Good', 'Premium', 'Ideal']
color_order = ['D', 'E', 'F', 'G', 'H', 'I', 'J']
clarity_order = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF']

diamonds_ordinal = diamonds.copy()
ordinal_encoder = OrdinalEncoder(categories=[cut_order, color_order,
↳clarity_order])
diamonds_ordinal[categorical_cols] = ordinal_encoder.
↳fit_transform(diamonds[categorical_cols])

print(diamonds_ordinal[categorical_cols].head(10))
```

	cut	color	clarity
0	4.0	1.0	1.0
1	3.0	1.0	2.0
2	1.0	1.0	4.0
3	3.0	5.0	3.0
4	1.0	6.0	1.0
5	2.0	6.0	5.0
6	2.0	5.0	6.0
7	2.0	4.0	2.0
8	0.0	1.0	3.0
9	2.0	4.0	4.0

```
[18]: diamonds_ordinal.head()
```

```
[18]:
```

	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	4.0	1.0	1.0	61.5	55.0	326	3.95	3.98	2.43
1	0.21	3.0	1.0	2.0	59.8	61.0	326	3.89	3.84	2.31
2	0.23	1.0	1.0	4.0	56.9	65.0	327	4.05	4.07	2.31
3	0.29	3.0	5.0	3.0	62.4	58.0	334	4.20	4.23	2.63

```
4    0.31  1.0    6.0    1.0   63.3   58.0   335  4.34  4.35  2.75
```

8. Apliquem normalització de dades:

```
[19]: scaler = StandardScaler()
diamonds_ordinal = scaler.fit_transform(diamonds_ordinal)
diamonds_ordinal = pd.DataFrame(diamonds_ordinal, columns=diamonds.columns)
diamonds_ordinal.head()
```

```
[19]:      carat      cut      color      clarity      depth      table      price \
0 -1.198168  0.981473 -0.937163 -1.245215 -0.174092 -1.099672 -0.904095
1 -1.240361  0.085889 -0.937163 -0.638095 -1.360738  1.585529 -0.904095
2 -1.198168 -1.705279 -0.937163  0.576145 -3.385019  3.375663 -0.903844
3 -1.071587  0.085889  1.414272 -0.030975  0.454133  0.242928 -0.902090
4 -1.029394 -1.705279  2.002131 -1.245215  1.082358  0.242928 -0.901839
```

```
      x      y      z
0 -1.587837 -1.536196 -1.571129
1 -1.641325 -1.658774 -1.741175
2 -1.498691 -1.457395 -1.741175
3 -1.364971 -1.317305 -1.287720
4 -1.240167 -1.212238 -1.117674
```

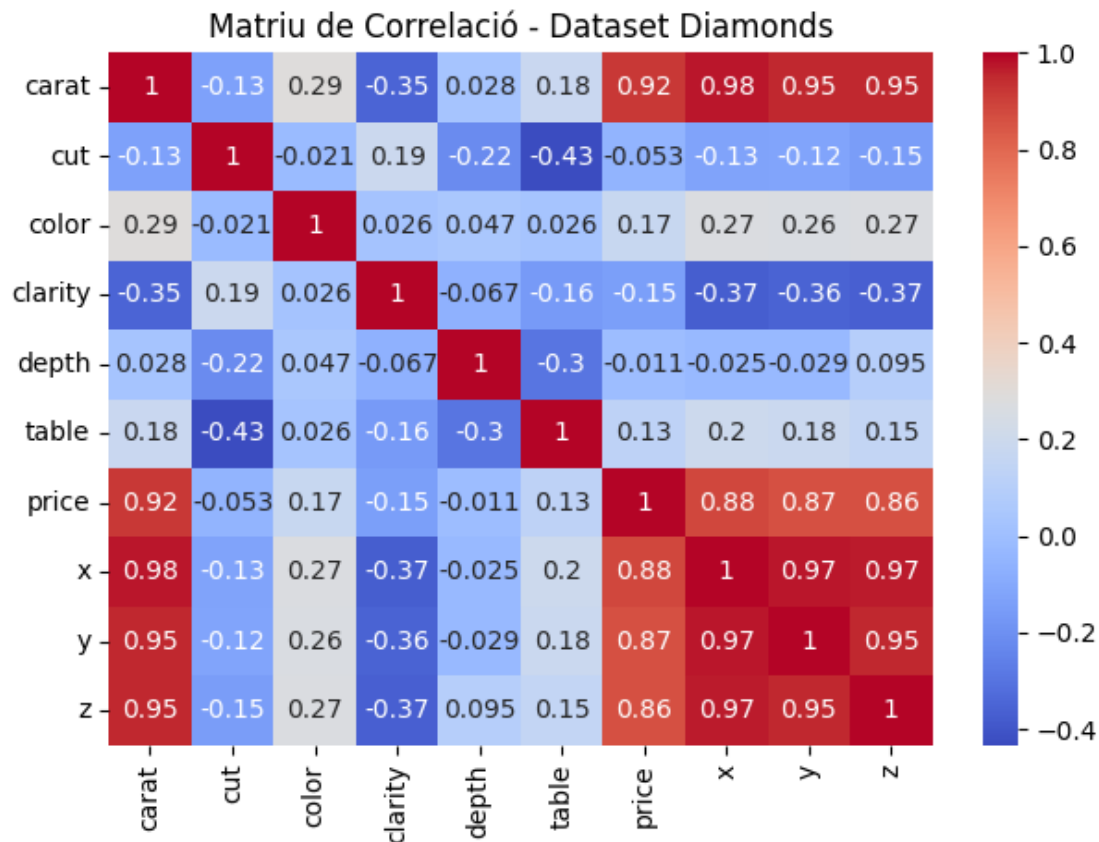
```
[20]: correlation_matrix = diamonds_ordinal.corr()
print(correlation_matrix)
```

```
      carat      cut      color      clarity      depth      table      price \
carat      1.000000 -0.134967  0.291437 -0.352841  0.028224  0.181618  0.921591
cut      -0.134967  1.000000 -0.020519  0.189175 -0.218055 -0.433405 -0.053491
color      0.291437 -0.020519  1.000000  0.025631  0.047279  0.026465  0.172511
clarity -0.352841  0.189175  0.025631  1.000000 -0.067384 -0.160327 -0.146800
depth      0.028224 -0.218055  0.047279 -0.067384  1.000000 -0.295779 -0.010647
table      0.181618 -0.433405  0.026465 -0.160327 -0.295779  1.000000  0.127134
price      0.921591 -0.053491  0.172511 -0.146800 -0.010647  0.127134  1.000000
x          0.975094 -0.125565  0.270287 -0.371999 -0.025289  0.195344  0.884435
y          0.951722 -0.121462  0.263584 -0.358420 -0.029341  0.183760  0.865421
z          0.953387 -0.149323  0.268227 -0.366952  0.094924  0.150929  0.861249
```

```
      x      y      z
carat      0.975094  0.951722  0.953387
cut      -0.125565 -0.121462 -0.149323
color      0.270287  0.263584  0.268227
clarity -0.371999 -0.358420 -0.366952
depth      -0.025289 -0.029341  0.094924
table      0.195344  0.183760  0.150929
price      0.884435  0.865421  0.861249
x          1.000000  0.974701  0.970772
y          0.974701  1.000000  0.952006
```


z 0.970772 0.952006 1.000000

```
[21]: sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Matriu de Correlació - Dataset Diamonds')
plt.tight_layout()
plt.show()
```



Obtenim els mateixos resultats: Les variables categòriques no eren rellevants.

9. Provem amb la regressió Polinòmica de diferents models Perquè sigui fàcil de calcular agafarem els 'best features'

```
[22]: best_features = ['carat', 'x']
X_numeric = diamonds[best_features]
y = diamonds['price']

X_train, X_test, y_train, y_test = train_test_split(
    X_numeric, y, test_size=0.2, random_state=42
)
```

```
[23]: # Crear features polinòmiques
poly = PolynomialFeatures(degree=2, include_bias=False)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
feature_names_poly = poly.get_feature_names_out(best_features)
def optim_alpha(model_class, alphas, X_poly, y_train):
    model_with_iter = model_class(max_iter=10000)
    param_grid = {'alpha': alphas}
    grid = GridSearchCV(model_with_iter, param_grid, cv=5, scoring='r2',
    ↪n_jobs=-1)
    grid.fit(X_poly, y_train)
    return grid.best_params_['alpha'], grid.best_score_

alphas = [0.01, 0.05, 0.1, 0.5, 1.0, 5.0, 10.0]

alpha_lasso, r2_lasso = optim_alpha(Lasso, alphas, X_train_poly, y_train)
alpha_ridge, r2_ridge = optim_alpha(Ridge, alphas, X_train_poly, y_train)
alpha_elastic, r2_elastic = optim_alpha(ElasticNet, alphas, X_train_poly,
    ↪y_train)

print(f"Lasso:  ={alpha_lasso} (R²={r2_lasso:.3f})")
print(f"Ridge:  ={alpha_ridge} (R²={r2_ridge:.3f})")
print(f"ElasticNet:  ={alpha_elastic} (R²={r2_elastic:.3f})")

Lasso:  =1.0 (R²=0.865)
Ridge:  =10.0 (R²=0.865)
ElasticNet:  =0.01 (R²=0.862)
```

```
[24]: alphas_dict = {
    'Lasso': alpha_lasso,
    'Ridge': alpha_ridge,
    'ElasticNet': alpha_elastic
}

models = {
    'Lasso': Lasso(alpha=alpha_lasso, max_iter=5000, random_state=42),
    'Ridge': Ridge(alpha=alpha_ridge, random_state=42),
    'ElasticNet': ElasticNet(alpha=alpha_elastic, l1_ratio=0.5, max_iter=5000,
    ↪random_state=42)
}

results = {}
for name, model in models.items():
    model.fit(X_train_poly, y_train)
    coefs = model.coef_
    nonzero = sum(coefs != 0)
```

```

results[name] = {'model': model, 'coefs': coefs, 'nonzero': nonzero}

print(f"\n{name}: {nonzero}/9 features")
coef_df = pd.DataFrame({
    'Feature': feature_names_poly,
    f'{name}_Coef': np.round(coefs, 3),
    f'{name}_Abs': np.round(np.abs(coefs), 3)
}).sort_values(f'{name}_Abs', ascending=False)
print(coef_df.to_string(index=False))

```

Lasso: 5/9 features

Feature	Lasso_Coef	Lasso_Abs
x	-5161.928	5161.928
carat ²	-4189.507	4189.507
carat x	2116.111	2116.111
carat	1312.778	1312.778
x ²	330.939	330.939

Ridge: 5/9 features

Feature	Ridge_Coef	Ridge_Abs
x	-5177.869	5177.869
carat ²	-4135.218	4135.218
carat x	2042.033	2042.033
carat	1992.733	1992.733
x ²	326.424	326.424

ElasticNet: 5/9 features

Feature	ElasticNet_Coef	ElasticNet_Abs
carat ²	-2133.774	2133.774
x	-2096.997	2096.997
carat x	1532.130	1532.130
carat	488.296	488.296
x ²	129.162	129.162

```

[25]: for name, res in results.items():
    model = res['model']

    # Prediccions
    y_pred_train = model.predict(X_train_poly)
    y_pred_test = model.predict(X_test_poly)

    print(f"\nModel {name} (={alphas_dict[name]:.2f}):")
    print(f"  R2 Train: {model.score(X_train_poly, y_train):.4f}")
    print(f"  R2 Test: {r2_score(y_test, y_pred_test):.4f}")
    print(f"  MSE Test: {mean_squared_error(y_test, y_pred_test):.2f}")
    print(f"  MAE Test: {mean_absolute_error(y_test, y_pred_test):.2f}")

```

```
print(f" Features: {res['nonzero']}/9 ({res['nonzero']/9*100:.0f}%)")
```

Model Lasso ($\lambda=1.00$):

R^2 Train: 0.8663

R^2 Test: 0.8696

MSE Test: 2072396.96

MAE Test: 847.50

Features: 5/9 (56%)

Model Ridge ($\lambda=10.00$):

R^2 Train: 0.8664

R^2 Test: 0.8695

MSE Test: 2074801.30

MAE Test: 849.05

Features: 5/9 (56%)

Model ElasticNet ($\lambda=0.01$):

R^2 Train: 0.8621

R^2 Test: 0.8644

MSE Test: 2156176.92

MAE Test: 858.92

Features: 5/9 (56%)